



Write a Python program to manage the borrowing records of books in a library.

```
from collections import Counter
```

```
class LibraryManager:
```

```
    def __init__(self):
```

```
        # Maps member_name -> number_of_books_borrowed
```

```
        self.member_records = {}
```

```
        # Maps book_title -> number_of_times_borrowed
```

```
        self.book_records = {}
```

```
    def add_member_record(self, member_name: str, borrow_count: int):
```

```
        """Adds or updates a library member's total borrowing count."""
```

```
        self.member_records[member_name] = borrow_count
```

```
    def add_book_record(self, book_title: str, borrow_count: int):
```

```
        """Adds or updates a specific book's total borrowing frequency."""
```

```
        self.book_records[book_title] = borrow_count
```

```
    def compute_average_borrowed(self) -> float:
```

```
        """Computes the average number of books borrowed per member."""
```

```
        if not self.member_records:
```

```
            return 0.0
```

```
        total_books = sum(self.member_records.values())
```

```
        return total_books / len(self.member_records)
```

```
    def find_highest_and_lowest_books(self) -> tuple:
```

```
        """Finds the books with the highest and lowest borrowings."""
```

```
        if not self.book_records:
```

```
            return None, None
```

```
        highest_book = max(self.book_records, key=self.book_records.get)
```

```
        lowest_book = min(self.book_records, key=self.book_records.get)
```

```
        return highest_book, lowest_book
```

```
    def count_zero_borrowers(self) -> int:
```

```
        """Counts members who have not borrowed any books (borrow count == 0)."""
```

```
        return sum(1 for count in self.member_records.values() if count == 0)
```

```
    def display_mode_borrow_counts(self) -> list:
```

```
        """Finds the most frequent borrow count(s) among members (Mode)."""
```

```
        if not self.member_records:
```

```
            return []
```

```
        counts = list(self.member_records.values())
```

```
        counter = Counter(counts)
```

```
        highest_frequency = max(counter.values())
```

```
        # Extract all counts that share the maximum frequency
```

```
        modes = [num for num, freq in counter.items() if freq == highest_frequency]
```

```
        return modes
```

KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU,
DTE Code: EN6634)

```
# --- Demonstration of the script ---
if __name__ == "__main__":
    library = LibraryManager()

    # Populate Sample Member Records (Member Name: Borrow Count)
    library.add_member_record("Alice", 3)
    library.add_member_record("Bob", 0)
    library.add_member_record("Charlie", 5)
    library.add_member_record("Diana", 0)
    library.add_member_record("Evan", 3)

    # Populate Sample Book Records (Book Title: Times Borrowed)
    library.add_book_record("The Great Gatsby", 12)
    library.add_book_record("1984", 25)
    library.add_book_record("To Kill a Mockingbird", 7)
    library.add_book_record("Moby Dick", 2)

    # 1. Compute average books borrowed by all members
    avg_borrowed = library.compute_average_borrowed()
    print(f"Average books borrowed per member: {avg_borrowed:.2f}")

    # 2. Find highest and lowest borrowed books
    highest, lowest = library.find_highest_and_lowest_books()
    print(f"Highest borrowed book: '{highest}' ({library.book_records[highest]} times)")
    print(f"Lowest borrowed book: '{lowest}' ({library.book_records[lowest]} times)")

    # 3. Count members with 0 borrowings
    zero_borrowers = library.count_zero_borrowers()
    print(f"Number of members with 0 borrowings: {zero_borrowers}")

    # 4. Display the mode of member borrow counts
    modes = library.display_mode_borrow_counts()
    print(f"Most frequent member borrow count(s) (Mode): {modes}")
```



In an e-commerce system, customer account IDs are stored in a list, and you are tasked with writing a program that implements the following:

```
class CustomerSearchManager:
    def __init__(self, account_ids: list):
        # Linear search works on an unsorted copy
        self.unsorted_ids = list(account_ids)
        # Binary search requires sorted data
        self.sorted_ids = sorted(account_ids)

    def linear_search(self, target_id: str) -> int:
        """
        Scans elements sequentially.
        Time Complexity: O(n)
        Returns the index if found, else -1.
        """
        for index, account_id in enumerate(self.unsorted_ids):
            if account_id == target_id:
                return index
        return -1

    def binary_search(self, target_id: str) -> int:
        """
        Divides the search space in half each step.
        Time Complexity: O(log n)
        Returns the index in the sorted list if found, else -1.
        """
        low = 0
        high = len(self.sorted_ids) - 1

        while low <= high:
            mid = (low + high) // 2
            mid_val = self.sorted_ids[mid]

            if mid_val == target_id:
                return mid
            elif mid_val < target_id:
                low = mid + 1
            else:
                high = mid - 1

        return -1

# --- Demonstration of the system ---
if __name__ == "__main__":
    # Sample list of e-commerce customer account IDs
    raw_customer_ids = ["ACC-904", "ACC-102", "ACC-550", "ACC-312", "ACC-789"]

    search_system = CustomerSearchManager(raw_customer_ids)
    target = "ACC-550"
```

KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU,
DTE Code: EN6634)

```
print(f"Original ID List: {search_system.unsorted_ids}")
print(f"Sorted ID List: {search_system.sorted_ids}\n")

# 1. Linear Search Execution
linear_result = search_system.linear_search(target)
if linear_result != -1:
    print(f"[Linear Search] Found '{target}' at index {linear_result} of the original list.")
else:
    print(f"[Linear Search] ID '{target}' not found.")

# 2. Binary Search Execution
binary_result = search_system.binary_search(target)
if binary_result != -1:
    print(f"[Binary Search] Found '{target}' at index {binary_result} of the sorted list.")
else:
    print(f"[Binary Search] ID '{target}' not found.")
```



In a company, employee salaries are stored in a list as floating-point numbers. Write a Python program that sorts the employee salaries in ascending order using the following two algorithms:

```
def merge_sort_salaries(salaries):
    """
    Sorts employee salaries in ascending order using Merge Sort.
    """
    # Base case: a list of 0 or 1 element is already sorted
    if len(salaries) <= 1:
        return salaries

    # Divide: Find the midpoint and split the array
    mid = len(salaries) // 2
    left_half = merge_sort_salaries(salaries[:mid])
    right_half = merge_sort_salaries(salaries[mid:])

    # Conquer: Merge the sorted halves
    return merge_salaries(left_half, right_half)
```

```
def merge_salaries(left, right):
    """
    Helper function to merge two sorted lists of salaries.
    """
    result = []
    i = j = 0

    # Compare elements from both lists and merge them in order
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
```

```
# Append any remaining elements from either list
result.extend(left[i:])
result.extend(right[j:])
return result
```

Example usage:

```
employee_salaries = [45000.50, 120000.75, 32000.00, 75000.25, 62000.00]
print("Original Salaries: ", employee_salaries)
print("Merge Sorted:    ", merge_sort_salaries(employee_salaries))
```



Implementing a real-time undo/redo system for a text editing application using a Stack data structure. The system should support the following operations:

```
class TextEditor:
    def __init__(self):
        self.current_text = ""
        self.undo_stack = [""] # Starts with an empty document state
        self.redo_stack = []

    def type_text(self, text: str):
        """Simulates appending text to the document."""
        self.current_text += text
        self.undo_stack.append(self.current_text)
        self.redo_stack.clear() # New actions invalidate previous redos
        print(f"Typed: '{text}' | Current: '{self.current_text}'")

    def undo(self):
        """Reverts to the previous text state."""
        if len(self.undo_stack) <= 1:
            print("Nothing to undo.")
            return

        # Move current state to redo stack
        popped_state = self.undo_stack.pop()
        self.redo_stack.append(popped_state)

        # Restore to the previous state
        self.current_text = self.undo_stack[-1]
        print(f"Undo action | Current: '{self.current_text}'")

    def redo(self):
        """Reapplies the last undone text state."""
        if not self.redo_stack:
            print("Nothing to redo.")
            return
```

```
# Pop from redo and push back to undo
restored_state = self.redo_stack.pop()
self.undo_stack.append(restored_state)

self.current_text = restored_state
print(f"Redo action | Current: '{self.current_text}'")
```




Implement a real-time event processing system using a Queue data structure. The system should support the following features:

```
import time
import threading
import queue
import random
from dataclasses import dataclass, field
from datetime import datetime

# 1. Define the Event Data Structure
@dataclass
class Event:
    event_id: str
    event_type: str
    payload: dict
    timestamp: datetime = field(default_factory=datetime.now)

# 2. Define the Real-Time Event Processor
class RealTimeEventProcessor:
    def __init__(self, num_workers=3):
        self.event_queue = queue.Queue()
        self.num_workers = num_workers
        self.workers = []
        self.is_running = False

    # Route events to specific logic based on type
    def _process_event(self, worker_id, event: Event):
        print(f"[Worker-{worker_id}] Processing {event.event_type} (ID: {event.event_id}) at {event.timestamp.isoformat()}")

        # Simulate business logic processing time
        if event.event_type == "PURCHASE":
            time.sleep(0.3) # Simulating DB write
            print(f"[Worker-{worker_id}] Success: Processed payment of ${event.payload.get('amount')}")
        elif event.event_type == "USER_SIGNUP":
            time.sleep(0.1) # Simulating welcome email
            print(f"[Worker-{worker_id}] Success: Created profile for {event.payload.get('user')}")
        else:
            time.sleep(0.05)
            print(f"[Worker-{worker_id}] Success: Logged system alert.")

    # Worker loop running in background threads
    def _worker_loop(self, worker_id):
        while self.is_running or not self.event_queue.empty():
            try:
                # Wait for an event with a timeout to check the running status periodically
                event = self.event_queue.get(timeout=1)
                self._process_event(worker_id, event)
                self.event_queue.task_done()
            except queue.Empty:
                continue

    # Public API to add events to the system
    def publish_event(self, event: Event):
        if not self.is_running:
```

KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU,
DTE Code: EN6634)

```
    print("System is offline. Cannot accept events.")
    return False
self.event_queue.put(event)
return True

# Start worker threads
def start(self):
    self.is_running = True
    print(f"Starting event processing system with {self.num_workers} workers...")
    for i in range(self.num_workers):
        t = threading.Thread(target=self._worker_loop, args=(i+1,), daemon=True)
        self.workers.append(t)
        t.start()

# Gracefully stop the system
def stop(self):
    print("\nInitiating graceful shutdown...")
    self.is_running = False # Signal workers to stop when queue is empty

    # Wait for all current tasks in the queue to be finished
    self.event_queue.join()

    # Wait for all threads to exit
    for t in self.workers:
        t.join()
    print("System stopped safely. No data lost.")

# 3. Simulate System Usage
if __name__ == "__main__":
    # Initialize processor with 3 parallel workers
    processor = RealTimeEventProcessor(num_workers=3)
    processor.start()

    # Simulate a stream of incoming events
    event_types = ["PURCHASE", "USER_SIGNUP", "SYSTEM_ALERT"]

    print("\n--- Simulating Live Traffic ---")
    for i in range(10):
        etype = random.choice(event_types)
        payload = (
            {"amount": random.randint(10, 500)} if etype == "PURCHASE"
            else {"user": f"user_{i}"} if etype == "USER_SIGNUP"
            else {"level": "WARN"}
        )

        evt = Event(event_id=f"EVT-{1000+i}", event_type=etype, payload=payload)
        processor.publish_event(evt)
        time.sleep(0.1) # Simulate real-time delay between user actions

    # Allow time for workers to catch up on final events before triggering shutdown
    time.sleep(2)
    processor.stop()
```



A call center receives incoming calls, and each call is assigned a unique customer ID. The calls are answered in the order they are received. Your task is to simulate the call queue of a call center using a queue data structure.

```
from collections import deque

class CallCenterQueue:
    def __init__(self):
        # Initialize an empty queue
        self.queue = deque()

    def receive_call(self, customer_id):
        """Adds a new call to the end of the queue."""
        self.queue.append(customer_id)
        print(f"📞 Call received from Customer ID: {customer_id}")

    def answer_call(self):
        """Answers the call at the front of the queue."""
        if not self.queue:
            print("🚫 No calls in the queue.")
            return None
        customer_id = self.queue.popleft()
        print(f"🎧 Answering Customer ID: {customer_id}")
        return customer_id

    def display_queue(self):
        """Shows all customers currently waiting."""
        print(f"🕒 Current Queue: {list(self.queue)}")

# --- Execution Simulation ---
if __name__ == "__main__":
    center = CallCenterQueue()

    # Simulate incoming calls
    center.receive_call("CUST-101")
    center.receive_call("CUST-102")
    center.receive_call("CUST-103")
    center.display_queue()

    # Simulate answering calls
    center.answer_call()
    center.answer_call()
    center.display_queue()

    # Simulate another incoming call
    center.receive_call("CUST-104")
    center.display_queue()
```



Create a Student Record Management System using linked list

- **Use a singly/doubly linked list to store student data (Roll No, Name, Marks).**
 - **Perform operations: Add, Delete, Update, Search, and Sort.**
- Display records in ascending/descending order based on marks or roll number.**

```
#include <iostream>
#include <string>

using namespace std;

// Node structure representing a student record
struct Student {
    int rollNo;
    string name;
    float marks;
    Student* next;

    // Constructor for easy initialization
    Student(int r, string n, float m) {
        rollNo = r;
        name = n;
        marks = m;
        next = nullptr;
    }
};

class StudentManagementSystem {
private:
    Student* head;

    // Helper function to check if a roll number already exists
    bool isDuplicate(int rollNo) {
        Student* temp = head;
        while (temp != nullptr) {
            if (temp->rollNo == rollNo) return true;
            temp = temp->next;
        }
        return false;
    }

public:
    StudentManagementSystem() {
        head = nullptr;
    }

    // 1. ADD RECORD (At the end of the list)
    void addRecord() {
        int r;
        string n;
        float m;
```

KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU,
DTE Code: EN6634)

```
cout << "\nEnter Roll Number: ";
cin >> r;
if (isDuplicate(r)) {
    cout << "Error: Student with Roll Number " << r << " already exists!\n";
    return;
}
```

```
cout << "Enter Name: ";
cin.ignore();
getline(cin, n);
cout << "Enter Marks: ";
cin >> m;
```

```
Student* newStudent = new Student(r, n, m);
```

```
if (head == nullptr) {
    head = newStudent;
} else {
    Student* temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
    temp->next = newStudent;
}
cout << "Record added successfully!\n";
}
```

```
// 2. DELETE RECORD by Roll Number
```

```
void deleteRecord() {
    if (head == nullptr) {
        cout << "\nDatabase is empty! Nothing to delete.\n";
        return;
    }
}
```

```
int r;
cout << "\nEnter Roll Number to delete: ";
cin >> r;
```

```
Student* temp = head;
Student* prev = nullptr;
```

```
// If head node itself holds the record to be deleted
```

```
if (temp != nullptr && temp->rollNo == r) {
    head = temp->next;
    delete temp;
    cout << "Record deleted successfully!\n";
    return;
}
```

```
// Search for the record to be deleted
```

```
while (temp != nullptr && temp->rollNo != r) {
    prev = temp;
    temp = temp->next;
}
```

KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU,
DTE Code: EN6634)

```
// If record was not found
if (temp == nullptr) {
    cout << "Record not found!\n";
    return;
}

// Unlink the node from linked list
prev->next = temp->next;
delete temp;
cout << "Record deleted successfully!\n";
}

// 3. UPDATE RECORD
void updateRecord() {
    if (head == nullptr) {
        cout << "\nDatabase is empty!\n";
        return;
    }

    int r;
    cout << "\nEnter Roll Number to update: ";
    cin >> r;

    Student* temp = head;
    while (temp != nullptr) {
        if (temp->rollNo == r) {
            cout << "Current Details -> Name: " << temp->name << ", Marks: " << temp->marks << "\n";
            cout << "Enter New Name: ";
            cin.ignore();
            getline(cin, temp->name);
            cout << "Enter New Marks: ";
            cin >> temp->marks;
            cout << "Record updated successfully!\n";
            return;
        }
        temp = temp->next;
    }
    cout << "Record not found!\n";
}

// 4. SEARCH RECORD
void searchRecord() {
    if (head == nullptr) {
        cout << "\nDatabase is empty!\n";
        return;
    }

    int r;
    cout << "\nEnter Roll Number to search: ";
    cin >> r;
```

KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU,
DTE Code: EN6634)

```
Student* temp = head;
while (temp != nullptr) {
    if (temp->rollNo == r) {
        cout << "\n--- Student Found ---" << endl;
        cout << "Roll No: " << temp->rollNo << endl;
        cout << "Name : " << temp->name << endl;
        cout << "Marks : " << temp->marks << endl;
        return;
    }
    temp = temp->next;
}
cout << "Record not found!\n";
}
```

// 5. SORT RECORDS (Using Bubble Sort logic on linked list nodes data swap)

```
void sortRecords(int criteria, bool ascending) {
    if (head == nullptr || head->next == nullptr) {
        return; // 0 or 1 element is already sorted
    }

    bool swapped;
    Student* ptr1;
    Student* lptr = nullptr;

    do {
        swapped = false;
        ptr1 = head;

        while (ptr1->next != lptr) {
            bool condition = false;

            if (criteria == 1) { // Sort by Roll Number
                condition = ascending ? (ptr1->rollNo > ptr1->next->rollNo) : (ptr1->rollNo < ptr1->next->rollNo);
            } else if (criteria == 2) { // Sort by Marks
                condition = ascending ? (ptr1->marks > ptr1->next->marks) : (ptr1->marks < ptr1->next->marks);
            }

            if (condition) {
                // Swap data
                swap(ptr1->rollNo, ptr1->next->rollNo);
                swap(ptr1->name, ptr1->next->name);
                swap(ptr1->marks, ptr1->next->marks);
                swapped = true;
            }
            ptr1 = ptr1->next;
        }
        lptr = ptr1;
    } while (swapped);
}
```

// 6. DISPLAY ALL RECORDS

```
void displayRecords() {
    if (head == nullptr) {
        cout << "\nNo records found to display.\n";
        return;
    }
}
```

KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU,
DTE Code: EN6634)

```
cout << "\n-----\n";
cout << "Roll No\t\tName\t\t\tMarks\n";
cout << "-----\n";
Student* temp = head;
while (temp != nullptr) {
    cout << temp->rollNo << "\t\t" << temp->name << "\t\t\t" << temp->marks << "\n";
    temp = temp->next;
}
cout << "-----\n";
}
};
```

```
int main() {
    StudentManagementSystem sms;
    int choice;

    while (true) {
        cout << "\n===== STUDENT RECORD MANAGEMENT SYSTEM =====";
        cout << "\n1. Add Student Record";
        cout << "\n2. Delete Student Record";
        cout << "\n3. Update Student Record";
        cout << "\n4. Search Student Record";
        cout << "\n5. Display All Records";
        cout << "\n6. Display Sorted Records (Ascending/Descending)";
        cout << "\n7. Exit";
        cout << "\nEnter your choice (1-7): ";
        cin >> choice;

        switch (choice) {
            case 1:
                sms.addRecord();
                break;
            case 2:
                sms.deleteRecord();
                break;
            case 3:
                sms.updateRecord();
                break;
            case 4:
                sms.searchRecord();
                break;
            case 5:
                sms.displayRecords();
                break;
            case 6: {
                int criteria, order;
                cout << "\nSort by:\n1. Roll Number\n2. Marks\nChoice (1-2): ";
                cin >> criteria;
                cout << "Order:\n1. Ascending\n2. Descending\nChoice (1-2): ";
                cin >> order;

                if((criteria == 1 || criteria == 2) && (order == 1 || order == 2)) {
                    sms.sortRecords(criteria, order == 1);
                    cout << "\nRecords sorted successfully!";
                    sms.displayRecords();
                } else {
                    cout << "Invalid sorting options selected.\n";
                }
            }
            case 7:
                return 0;
        }
    }
}
```


KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU,
DTE Code: EN6634)

```
    }  
        break;  
    }  
    case 7:  
        cout << "\nExiting System. Goodbye!\n";  
        return 0;  
    default:  
        cout << "\nInvalid Choice! Please enter a number between 1 and 7.\n";  
    }  
}  
return 0;  
}
```

KJ's Educational Institute
TRINITY ACADEMY OF ENGINEERING, PUNE
(Approved by AICTE, New Delhi, Govt. of Maharashtra & affiliated to SPPU,
DTE Code: EN6634)